AWS DynamoDB Fundamentals

Simple concept guide & quick refresher — based on the completed NextWork project

1. What is Amazon DynamoDB?

Simple meaning: Amazon DynamoDB is a non-relational (NoSQL) database service. Instead of requiring a traditional fixed row-and-column structure, it organizes data using **items** and **attributes**.

Purpose: Store and retrieve application data in a flexible database structure.

Why use it: It is useful when different items may need different attributes and when fast access to items using partition keys is important.

Simple example: An online store could have products where one item has *Size* and another has *ScreenSize*. DynamoDB allows items to have different attributes.

2. DynamoDB Table

Simple meaning: A DynamoDB table is a container for related items. The project describes DynamoDB tables as using items and attributes rather than the traditional relational-table structure.

Purpose: Provide a place to store and manage data.

Example: A table called **NextWorkStudents** can contain a student item such as **Nikko** with an attribute such as **ProjectsComplete = 4**.

3. Item

Simple meaning: An item is one individual record/object stored in a DynamoDB table.

Purpose: Represent one thing you want to store, such as a student, project, product, or video.

Example: Nikko can be one item in the NextWorkStudents table.

4. Attribute

Simple meaning: An attribute is a piece of data about an item.

Purpose: Describe an item with information that belongs to it.

Example: For Nikko, **ProjectsComplete = 4** is an attribute.

Key idea: DynamoDB items can have different sets of attributes. Adding **StudentsComplete** to one item does not automatically add it to every other item.

5. Partition Key

Simple meaning: The partition key is the key DynamoDB uses to organize and efficiently find items.

Purpose: Give DynamoDB a way to identify and locate data efficiently.

Important: Every item must have a value for the table's partition key.

Example: If **StudentName** is the partition key, DynamoDB can use a student's name when finding that student's item.

6. Sort Key

Simple meaning: In the project, a sort key is used together with a partition key in tables such as **Post** and **Comment**.

Example from the project: Post uses **ForumName** as the partition key and **Subject** as the sort key. Comment uses **Id** as the partition key and **CommentDateTime** as the sort key.

7. Read Capacity Units (RCUs)

Simple meaning: RCUs measure the read activity a DynamoDB table can handle per second.

Project rule: The project uses low provisioned capacity and sets read capacity to **1** for the Free Tier-focused setup.

Simple mental model: Think of RCUs as the read-processing capacity available to the table.

8. Write Capacity Units (WCUs)

Simple meaning: WCUs measure the write activity a DynamoDB table can handle per second.

Project rule: The project sets write capacity to **1** and turns off auto scaling for its low-cost setup.

Simple mental model: Think of WCUs as the write/update/delete processing capacity available to the table.

9. Auto Scaling

Simple meaning: Auto scaling can automatically adjust database performance based on demand.

Why it was turned off: The project wanted the table to stay at the low, controlled capacity settings used for the Free Tier-focused setup. Unmonitored scaling could increase processing capacity.

Exam/Interview idea: Auto scaling = automatically adjust capacity as demand changes.

10. AWS CloudShell

Simple meaning: AWS CloudShell is a browser-based shell environment inside the AWS Management Console.

Purpose: Give you a command-line environment for running commands that manage AWS resources.

Why use it: The AWS CLI is already available in CloudShell, so you can manage resources without separately installing the CLI on your computer.

Example: The project used CloudShell to run DynamoDB commands that created four tables.

11. AWS CLI

Simple meaning: AWS CLI is a command-line tool for creating, managing, and interacting with AWS resources using commands instead of the AWS Management Console.

Purpose: Manage AWS resources efficiently through commands and scripts.

Example: aws dynamodb create-table creates a DynamoDB table from the command line.

12. Creating DynamoDB Tables with CLI

The project used AWS CLI commands to create four tables:

- **ContentCatalog** — partition key: Id
- **Forum** — partition key: Name
- **Post** — partition key: ForumName; sort key: Subject
- **Comment** — partition key: Id; sort key: CommentDateTime

Why use CLI here? Creating resources through commands is faster and easier to repeat than manually creating many resources in the console.

13. Wait Command

Simple meaning: A wait command tells the CLI to keep waiting until a specified condition is met.

Purpose: Confirm that a resource is ready before continuing with commands that depend on it.

Example: aws dynamodb wait table-exists --table-name ContentCatalog waits until the table exists.

14. JSON Data Files & batch-write-item

Simple meaning: The project used JSON files containing DynamoDB-formatted data and loaded them with the AWS CLI.

batch-write-item: A CLI operation used in the project to insert multiple items into DynamoDB.

Example: aws dynamodb batch-write-item --request-items file://Forum.json loads the requests stored in Forum.json into the Forum table.

15. Unprocessed Items

Simple meaning: Unprocessed items are records that were not written to the database during a batch write.

Purpose: The response helps you notice that some data was not successfully written.

What to do: If unprocessed items appear, check the file/error, correct the data if necessary, and run the batch-write command again.

16. Flexibility vs Relational Databases

DynamoDB: Each item can have its own unique set of attributes.

Relational database: Rows normally follow the table's defined columns.

Simple example: One ContentCatalog item can contain project-related attributes such as Difficulty and ProjectCategory, while another item can contain video-related attributes such as VideoType and Services.

Why it matters: DynamoDB's flexible item structure is useful when different types of data need different attributes.

17. Speed / Fast Access

Simple meaning: DynamoDB can use partition keys to quickly find items instead of scanning through the entire table.

Why it matters: Efficient key-based access is useful for applications that need fast retrieval of specific data.

18. Relational vs Non-Relational — When to Choose

DynamoDB / non-relational: Useful when flexible item structures and fast key-based access are important.

Relational databases: Useful when SQL and complex queries are important, or when a strict schema helps maintain precise and consistent data. The project gives healthcare records as an example.

19. Free Tier & Cost Awareness

The project is designed around a Free Tier-focused DynamoDB configuration and uses minimal provisioned capacity. The extracted project states a DynamoDB Free Tier allowance of **25 GB storage, 25 WCUs and 25 RCUs** and advises deleting resources when they are no longer needed to avoid charges.

Important: Free Tier limits and AWS pricing can change. Treat the figures above as the values stated in this project, not as a permanent pricing guarantee.

20. Quick Revision — Remember These

Concept	Remember
DynamoDB	AWS non-relational / NoSQL database
Table	Container for related DynamoDB items
Item	One individual record/object
Attribute	Piece of data about an item
Partition key	Used to organize and efficiently find items
Sort key	Additional key used with a partition key
RCU	Read capacity
WCU	Write capacity
CloudShell	Browser-based shell in AWS Console
AWS CLI	Commands for managing AWS resources
batch-write-item	Loads multiple items
Wait command	Waits until a condition is met
Flexibility	Items can have different attributes

One-line mental model: DynamoDB stores flexible items and attributes; partition keys help find data efficiently; RCUs/WCUs represent read/write capacity; CloudShell gives you a browser-based terminal; AWS CLI lets you manage AWS resources with commands.

Source basis: NextWork project extract provided by the user. This guide intentionally stays close to the project's terminology and explanations so it can be used as a revision document.